任务 5 分析

任务 5：文件功能模块：设计并实现网表结构，支持设计文件的导入导出功能。导出标准网表文件，该文件能够被 PCB 软件加载，进行后续设计流程。

1．基本思想

该任务分为如下几个子任务：

子任务 1：对设计的原理图进行保存和加载

子任务 2：导出标准网表文件

子任务 3：该文件能够被 PCB 软件加载，进行后续设计流程

因为不同的 PCB 软件，有不同的网表标准，如下将子任务 2 和子任务 3 联合来考虑。

我们定标用开源 gEDA 软件的 gschem（原理图程序）和 pcb（PCB 程序）来完成任务。gEDA 是一套开源电子设计自动化工具，其设计流程主要分为三步：

1） 使用 gschem 绘制原理图

2） 通过 gsch2pcb 生成网表

3) 使用 pcb 进行布局布线。

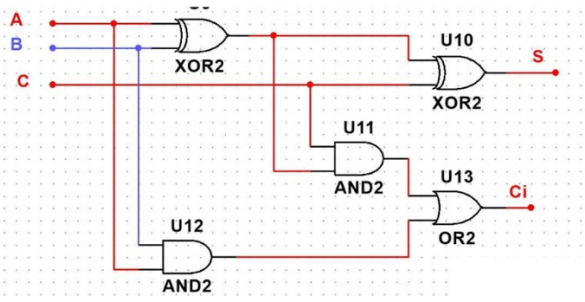
首先看看 gschem+pcb 是如何工作的，然后将自己的原理图程序替代 gschem。

2．gschem+pcb 过程

以一位全加器为例，在 gschem 中绘制原理图，之后在 pcb 绘制印制板。

2.1 全加器原理

一位全加器的电路图和真值表如下：



真值表

C	A	B	Ci	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

通过分析图中逻辑门的连接关系，可以验证其功能：

- 输入：A、B、C（C 为来自低位的进位输入）。
- 输出：S（本位和）、Ci（向高位的进位输出）。

1. 本位和 S (Sum)

电路左侧的 U1 (XOR2) 先计算 A ⊕ B。然后，其结果与 C 通过 U10 (XOR2) 再次进行异或运算，得到输出 S。

- 逻辑表达式：S = (A ⊕ B) ⊕ C。这正是全加器本位和的逻辑功能。

2. 进位输出 Ci (Carry Out)

进位输出 Ci 是或门 U13 (OR2) 的输出，其两个输入分别是：

- U11 (AND2) 的输出：(A ⊕ B) · C
- U12 (AND2) 的输出：A · B

因此，Ci 的逻辑表达式为：Ci = (A ⊕ B)C + AB。

- 根据逻辑代数，该表达式可以化简为 Ci = AB + BC + AC，这正是全加器进位输出的标准逻辑。

该电路使用了 2 个异或门（如 74HC86）、2 个与门（如 74HC08）和 1 个或门（如 74HC32），这些元件在开源软件中有标准件：

gEDA：

- 7409_4xAND：单路 2 输入与门 (AND)
- 7432_4xOR： 单路 2 输入或门 (OR)
- 7486_4xXOR：单路 2 输入异或门 (XOR)

Kicad：

- 74AHC1G08：单路 2 输入与门 (AND)
- 74AHC1G32：单路 2 输入或门 (OR)
- 74AHC1G86：单路 2 输入异或门 (XOR)

2.2 使用 gEDA 绘制全加器原理图

三部曲：在 gschem 中绘制全加器原理图 → 为每个逻辑门分配 RefDes（Reference Designator，参考编号） 与 Footprint（封装名）属性 → 使用 gsch2pcb 生成 PCB 可读的网表文件。

如下假设已经在 Linux 环境编译和安装了 gschem 与 pcb 软件，详细步骤见 WSL ubuntu24.04-gEDA 编译.pdf

2.2.1 在 gschem 中绘制全加器原理图

步骤 1：运行 gschem

```
a@g2:~$ gschem
```

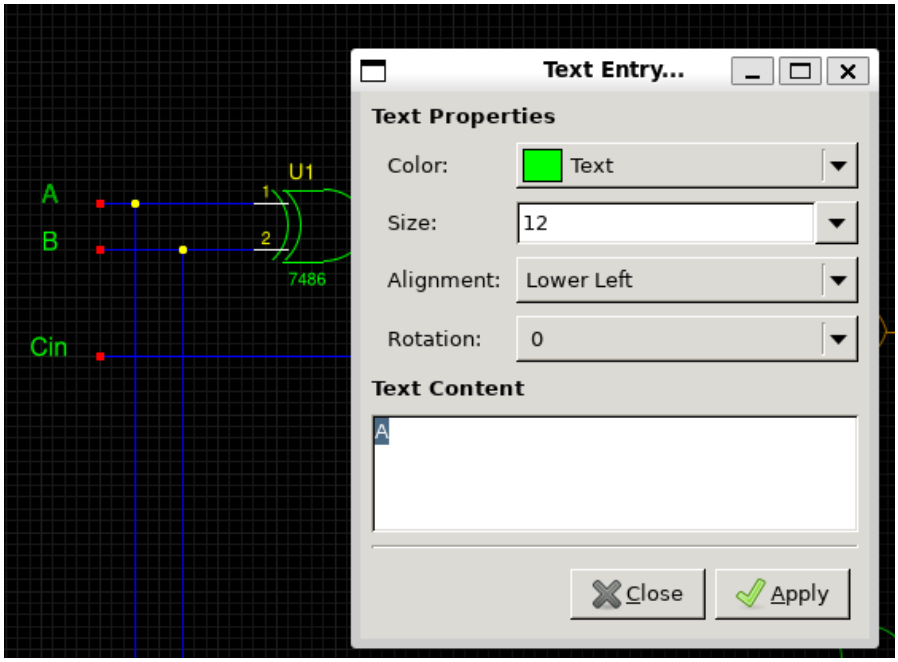
步骤 2：在程序界面中添加和放置元件

点击菜单 Add → Component（或按快捷键 i）打开符号库，搜索 74

依次放置 2 个 7486（xor2, 用于 Sum 路径）、1 个 7432（or2, 用于 Cout 路径），2 个 7409（and2 用于 Cout 路径）

步骤 3：添加标识

点右键 Add text，输入标识，点 Apply，将鼠标移动到相应位置，点击即可添加标识 A、B、C、S、Ci 等。



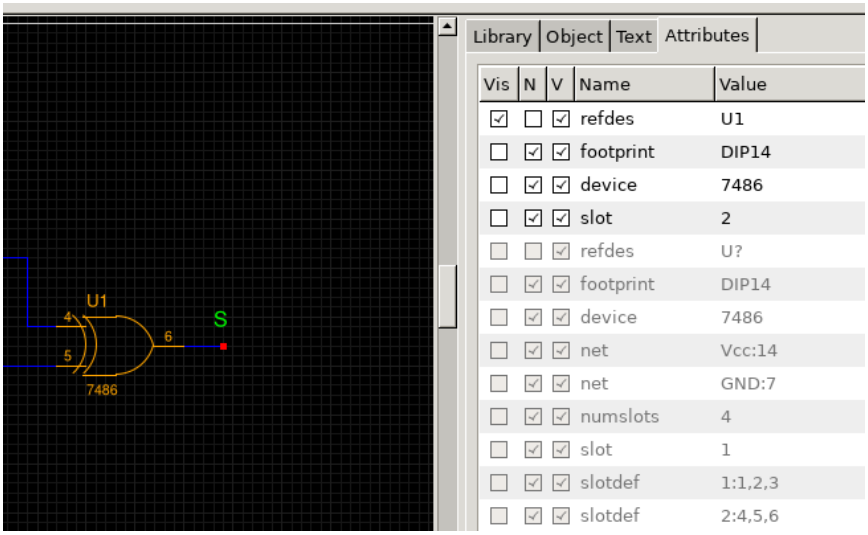
步骤 4：移动、旋转与连线

选中元件后，按 m 移动，er 旋转，ei 镜像，调整到合适布局。右键 add net 或（按 n） 切换到连线工具进行连线。

2.2.2 为每个逻辑门分配 RefDes 与 Footprint

这是让原理图“对接”PCB 的关键一步。PCB 工具不认图形，只认参考编号（RefDes，如 U1、U2、R1、C3）和封装名（Footprint，如 DIP14、SO14、TSSOP14）。

- RefDes 用来唯一标识一个物理元件实例。PCB 网表、BOM、丝印、调试时都靠它对应。例如第一个异或门设为 U1，第一个与门设为 U2 等。
- Footprint 可以填写你打算采用的封装。Footprint 是 PCB 库里的封装名，它告诉 PCB 工具：这个元件在 PCB 上要放 14 脚直插（DIP14）还是 14 脚贴片（SO14），焊盘间距多大。如果缺少 footprint 属性，后续工具将无法知道该用什么样的物理焊盘。



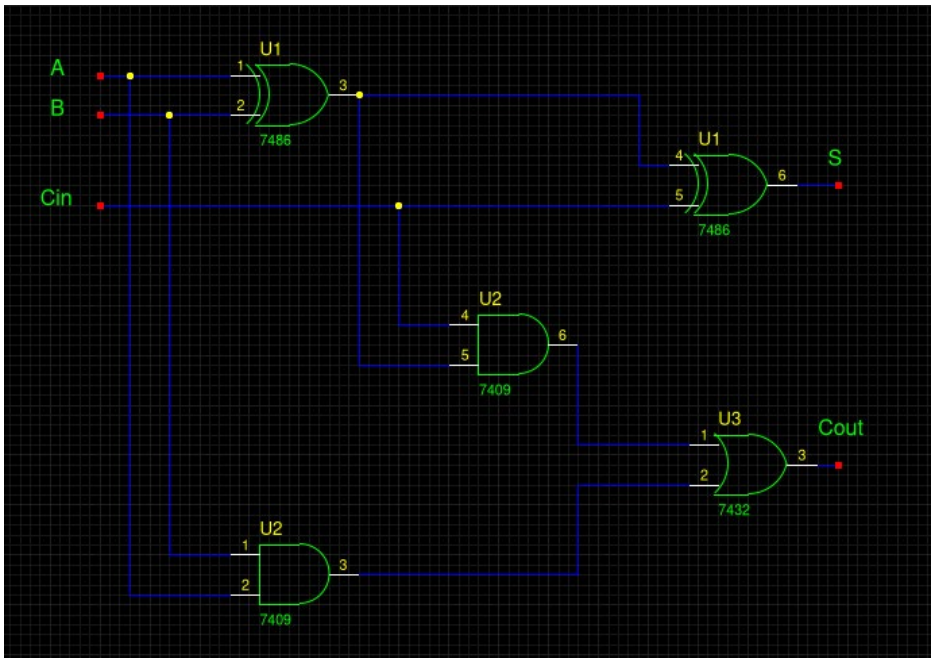
A. 典型分配方案

对全加器，可有如下的典型分配方案。

假设用三片 14 脚逻辑 IC：

原理图逻辑门	物理芯片	RefDes	Footprint	Slot/门
第 1 个 XOR	7486	U1	DIP14	slot 1
第 2 个 XOR	7486	U1	DIP14	slot 2
第 1 个 AND	7409	U2	DIP14	slot 1
第 2 个 AND	7409	U2	DIP14	slot 2
OR	7432	U3	DIP14	slot 1

注：7486 是四异或门：一颗 14 脚芯片里有 4 个独立的异或门，Slot 即指明是芯片中的第几个门。同一片多门 IC 的多个门，RefDes 必须相同，用 slot 区分门；不能每个门都给不同 U 编号，否则 PCB 会以为你有 5 片芯片。



2.2.3 生成 PCB 可读的网表文件

将当前原理图存盘保存为 full_adder.sch，

编辑生成编写项目文件 full_adder.prj，其内容为：

```
elements-dir ./footprints
schematics full_adder.sch
output-name full_adder
```

在终端，在其目录中执行：

gsch2pcb full_adder.prj

执行后，通常会在当前目录生成 2 个文件：

- full_adder.net（或类似名称）：标准的 PCB 格式网表，包含了所有元件及其引脚间的电气连接关系。
- full_adder.pcb：一个初步的 PCB 布局文件，其中已经按照你在原理图中填写的 footprint 属性，自动放置了对应的元件封装。
- full_adder.cmd：批处理更换引脚名的命令，若无需要，可以忽略该文件。

2.3. 使用 PCB 绘制全加器印制板

设无需更换引脚名，则 pcb 的输入为 full_adder.pcb 和 full_adder.net 两个文件。

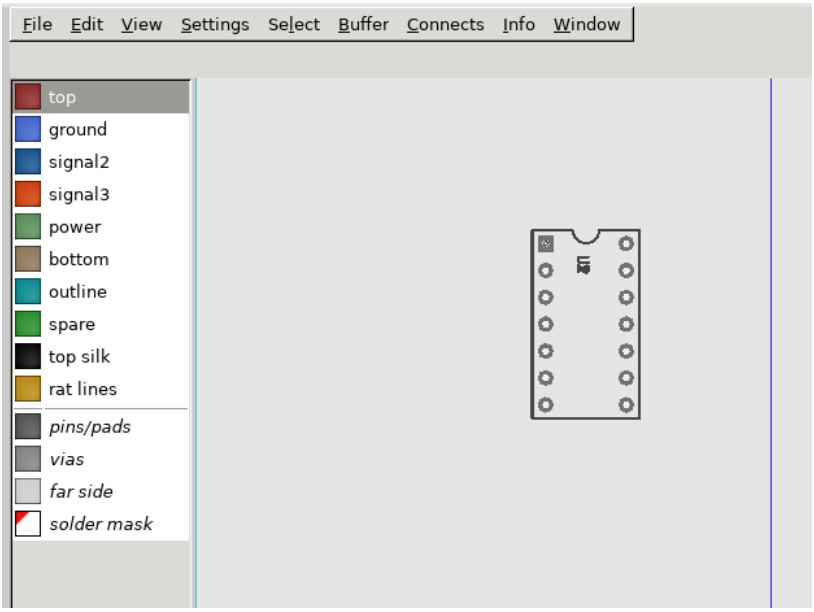
具体操作如下：

步骤 1：运行 pcb

```
a@g2:~/zs/bin$ pcb
```

步骤 2：打开 pcb 软件。执行 File→New 新建空白 PCB 工程，进入设计画布界面。

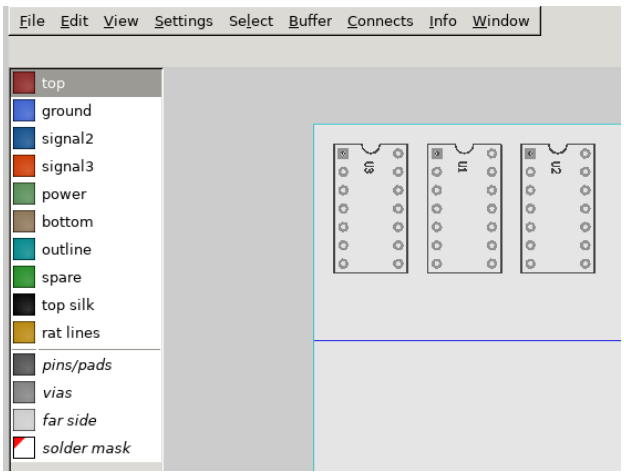
步骤 3：加载元件布局。执行菜单项“File→Load layout data to buffer”加载新的元件数据，在弹出的文件选择对话框中选择“full_adder.pcb”，读取后，full_adder.pcb 中的元件封装被放置到缓冲区，在电路板上的合适位置点击鼠标左键以放置这些新元件。由于原理图并不具备元件的实际位置，所以此时所有元件都位于鼠标所指的同一位置。



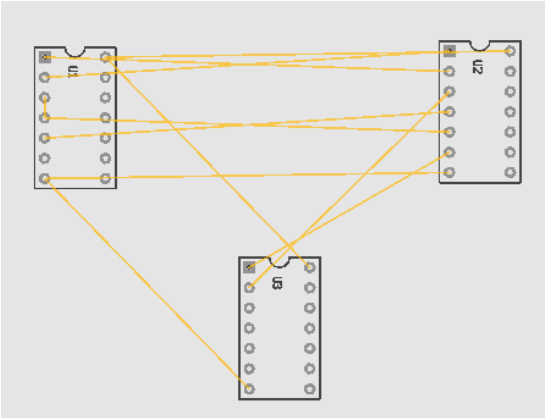
步骤 4：加载网表与引脚重命名脚本。通过“File→Load netlist”菜单项，选择“full_adder.net”文件来导入网表。

注：若需要批处理更换引脚名，按下“:”键调出命令行，执行“ExecuteFile(full_adder.cmd)”，完成引脚名

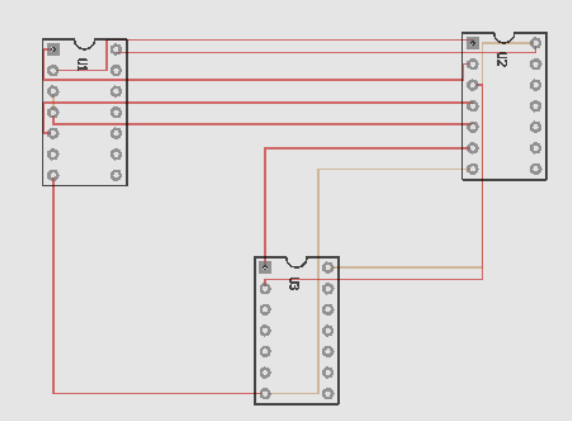
步骤 5：元件初步散布。执行“Select→Disperse all elements”菜单项，将重合的元件等距分散，便于后续布局调整。如图所示，Disperse 功能仅实现位置散开，展示了有 3 个元件。



步骤 6：将 3 个元件拖至一定位置，按‘o’键进行飞线，即按 full_adder.net 的内容进行逻辑连线。

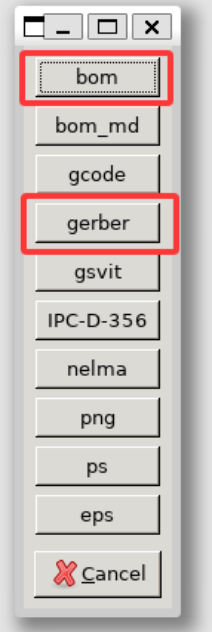


步骤 7：执行菜单项“Connects→Auto-route all rats”进行自动布线。



步骤 8：执行其他操作，如添加电源适配器、输出连接器 MTA100_3 等。

步骤 9：执行菜单项“File→Export…”，选择选项输出后续加工所需的结果：



选择 bom 生成元件文件 full_adder.bom 与其位置坐标 full_adder.xy 文件

选择 gerber 生成光刻文件 full_adder.bottom.gbr，full_adder.bottommask.gbr，full_adder.topmask.gbr，full_adder.topsilk.gbr 等文件。

2．自开发的原理图+pcb 过程

根据 gschem+pcb 过程可知，自开发的原理图需要输出如下标准文件给 pcb：

1）初始 pcb 文件 full_adder.pcb

2）网表文件 full_adder.net